

CLAVE DE RESPUESTAS — SOLO PARA USO DEL DOCENTE, NO REPARTIR

Métodos de programación — Clave

Examen — Primer parcial · Versión 3 (de 3)

1. Segundos a horas:minutos:segundos

Problema general:

El sistema de cronometraje de una competencia de robótica registra internamente la duración de cada intento únicamente como un número entero de segundos transcurridos, porque así es más simple para el microcontrolador llevar la cuenta. Sin embargo, en la pantalla que ven el público y los jueces, el tiempo debe mostrarse en el formato tradicional horas:minutos:segundos (por ejemplo, un intento de 9385 segundos debe verse como 2 horas, 36 minutos y 25 segundos), ya que algunos intentos de las categorías de resistencia pueden durar varias horas. Diseña el algoritmo que, dado el número total de segundos registrado por el cronómetro, calcule cuántas horas completas, minutos completos y segundos restantes deben mostrarse en pantalla.

Problema bien definido:

Dado $s \in \mathbb{Z}_{\geq 0}$, calcular h, m, r tales que $s = 3600h + 60m + r$, con $0 \leq m < 60$, $0 \leq r < 60$ y h máxima posible.

Entrada:

$s \in \mathbb{Z}_{\geq 0}$

Instancia del problema:

Prueba para $s = 9385$

Salida:

Terna (h, m, r)

Algoritmo paso a paso:

1. $h \leftarrow s \text{ div } 3600$
2. $\text{resto} \leftarrow s \text{ mod } 3600$
3. $m \leftarrow \text{resto} \text{ div } 60$
4. $r \leftarrow \text{resto} \text{ mod } 60$
5. Regresar (h, m, r)

2. Máximo común divisor

Problema general:

Un carpintero necesita cortar dos tablones de madera de longitudes distintas en trozos más pequeños, todos del mismo tamaño y sin que sobre ningún desperdicio de ninguno de los dos tablones. Para que el corte sea lo más eficiente posible (usando la menor cantidad de cortes), el tamaño de cada trozo debe ser lo más grande posible, es decir, debe ser el mayor número entero que divide exactamente a ambas longitudes de tablón. Este mismo problema aparece también al programar un sistema que necesita simplificar fracciones a su mínima expresión. Diseña el algoritmo que, dados dos números enteros positivos que representan las dos longitudes, determine el mayor número entero que divide exactamente a ambos, sin necesidad de probar uno por uno todos los divisores posibles de principio a fin (piensa en cómo el residuo de una división te puede ayudar a reducir el problema a uno más pequeño).

Problema bien definido:

Dados $a, b \in \mathbb{Z}^+$, calcular $d = \text{mcd}(a, b)$, el mayor entero que divide exactamente a ambos.

Entrada: $a, b \in \mathbb{Z}^+$ **Instancia del problema:**Prueba para $a = 252$, $b = 105$ **Salida:** $d \in \mathbb{Z}^+$ **Algoritmo paso a paso:**

1. Mientras $b \neq 0$:
 - $r \leftarrow a \bmod b$
 - $a \leftarrow b$
 - $b \leftarrow r$
 2. Regresar a
-

3. Eliminar duplicados**Problema general:**

El sistema de registro de asistencia de un evento masivo de la universidad escanea el código de cada gafete conforme los asistentes van llegando y va guardando esos códigos en una lista, en el orden en que fueron escaneados. Debido a que algunos asistentes distraídos pasan el gafete dos o más veces por el mismo lector, la lista termina teniendo códigos repetidos. Para generar el reporte final de asistentes únicos que se entrega a los organizadores, el sistema necesita producir una segunda lista que contenga cada código una sola vez, respetando el orden en que cada código apareció por primera vez en la lista original. Diseña el algoritmo que, dada la lista original de códigos (con posibles repeticiones), genere la lista de códigos únicos sin duplicados.

Problema bien definido:

Dado $A = [a_1, \dots, a_n]$, obtener $B = [b_1, \dots, b_m]$ ($m \leq n$) que contenga cada valor distinto de A exactamente una vez, preservando el orden de primera aparición.

Entrada: $A = [a_1, \dots, a_n]$ **Instancia del problema:**Prueba para $A = [4, 7, 4, 2, 7, 9, 2, 4]$ **Salida:** $B = [b_1, \dots, b_m]$ **Algoritmo paso a paso:**

1. $m \leftarrow 0$
2. Para i desde 1 hasta n :
 - encontrado $\leftarrow$ Falso
 - Para j desde 1 hasta m :
 - Si $B[j] = A[i]$:
 - encontrado $\leftarrow$ Verdadero
 - Si encontrado = Falso:
 - $m \leftarrow m + 1$

$$B[m] \leftarrow A[i]$$

3. Regresar B (sus primeros m elementos)

4. Fusión de dos listas ordenadas RETO — OPCIONAL

Problema general:

Dos profesores de la misma materia llevan cada uno, por separado, la lista de calificaciones finales de su respectivo grupo, y cada uno ya la tiene ordenada de menor a mayor porque así es como se las pidió la coordinación. Antes de la junta de academia, la coordinación necesita una sola lista combinada con las calificaciones de ambos grupos juntos, y esa lista combinada también debe quedar ordenada de menor a mayor, para poder calcular estadísticas generales de la materia sin tener que ordenar todo desde cero (lo cual sería un desperdicio, ya que las dos listas originales ya vienen ordenadas). Diseña el algoritmo que, dadas dos listas de números ya ordenadas de menor a mayor (posiblemente de tamaños distintos), construya una sola lista ordenada que contenga exactamente todos los elementos de ambas listas, aprovechando el hecho de que ambas ya llegan ordenadas (es decir, sin necesidad de volver a ordenar todo desde cero comparando cada elemento contra todos los demás).

Problema bien definido:

Dados $A = [a_1, \dots, a_n]$ y $B = [b_1, \dots, b_m]$, ambos ordenados de forma no decreciente, obtener $C = [c_1, \dots, c_{(n+m)}]$ ordenado que contenga exactamente los elementos de A y B.

Entrada:

$A = [a_1, \dots, a_n]$ ordenado, $B = [b_1, \dots, b_m]$ ordenado

Instancia del problema:

Prueba para $A = [2, 5, 9, 14]$, $B = [1, 6, 7, 20]$

Salida:

$C = [c_1, \dots, c_{(n+m)}]$

Algoritmo paso a paso:

1. $i \leftarrow 1$
2. $j \leftarrow 1$
3. $k \leftarrow 1$
4. Mientras $i \leq n$ Y $j \leq m$:
 - Si $A[i] \leq B[j]$:
 - $C[k] \leftarrow A[i]$
 - $i \leftarrow i + 1$
 - Si no:
 - $C[k] \leftarrow B[j]$
 - $j \leftarrow j + 1$
 - $k \leftarrow k + 1$
5. Mientras $i \leq n$:
 - $C[k] \leftarrow A[i]$
 - $i \leftarrow i + 1$
 - $k \leftarrow k + 1$
6. Mientras $j \leq m$:
 - $C[k] \leftarrow B[j]$
 - $j \leftarrow j + 1$

$k \leftarrow k + 1$

7. Regresar C
